

Software Engineering in Practice — Assignment 1 (2026)

Advanced DevOps: Production-Grade CI/CD, External Configuration, and Orchestration

Romain Gazon - 8230028

This PDF constitutes the submission for SEIP Assignment 1 and includes all required screenshots, along with the AI Reflection and Future Outlook sections for my project hosted on GitHub ([link](#)).

Screenshots

CI/CD Proof

All workflows

Showing runs from all workflows

3 workflow runs

Workflow ▾ Event ▾ Status ▾ Branch ▾ Actor ▾

✓ Add CMD to Dockerfile

CI-CD #3: Commit 8e5529d pushed by romgzn

main

May 22, 10:27 AM GMT+3

23s

...

✓ Fix old workflow

CI-CD #2: Commit a1ebcd7 pushed by romgzn

main

May 22, 10:23 AM GMT+3

21s

...

✓ Fix workflow folder name

CI-CD #1: Commit 0666e6e pushed by romgzn

main

May 22, 10:21 AM GMT+3

25s

...

Cluster State Proof

```
C:\Users\User\Desktop\ΟΠΑ_ΔΕΤ\6ο ΕΞΑΜΗΝΟ\Τεχνολογίες Λογισμικού\seip_assignment_1_2026>kubectl get all -n default
NAME                                READY    STATUS    RESTARTS    AGE
pod/echo-api-deployment-6dd7fb4497-96cqz    1/1      Running   0            21m
pod/echo-api-deployment-6dd7fb4497-dh4q5    1/1      Running   0            21m
pod/echo-api-deployment-6dd7fb4497-x5zv2    1/1      Running   0            21m

NAME                                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
service/echo-api-service            ClusterIP     10.105.205.53 <none>         80/TCP     20m
service/kubernetes                  ClusterIP     10.96.0.1     <none>         443/TCP    25h

NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
deployment.apps/echo-api-deployment  3/3      3              3            21m

NAME                                DESIRED    CURRENT    READY    AGE
replicaset.apps/echo-api-deployment-6dd7fb4497    3          3          3        21m

C:\Users\User\Desktop\ΟΠΑ_ΔΕΤ\6ο ΕΞΑΜΗΝΟ\Τεχνολογίες Λογισμικού\seip_assignment_1_2026>kubectl get configmap,secret
NAME                                DATA    AGE
configmap/echo-api-config           2        22m
configmap/kube-root-ca.crt          1        25h

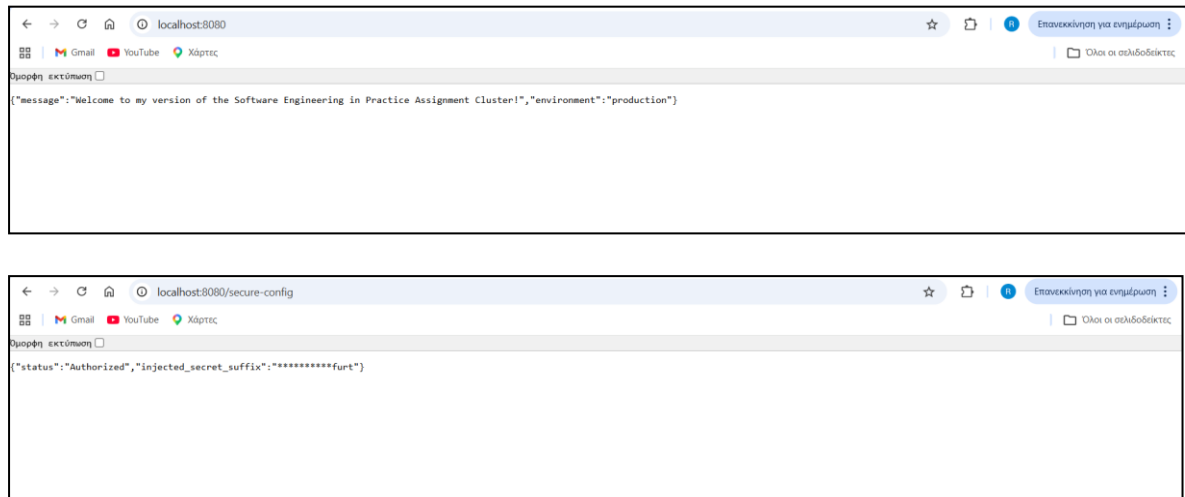
NAME                                TYPE    DATA    AGE
secret/echo-api-secret              Opaque  1        21m
```

Application Verification Proof

Terminal Response

```
C:\Users\User\Desktop\ΟΠΑ_ΔΕΤ\6ο ΕΞΑΜΗΝΟ\Τεχνολογίες Λογισμικού\seip_assignment_1_2026>curl http://localhost:8080/
{"message":"Welcome to my version of the Software Engineering in Practice Assignment Cluster!","environment":"production"}
C:\Users\User\Desktop\ΟΠΑ_ΔΕΤ\6ο ΕΞΑΜΗΝΟ\Τεχνολογίες Λογισμικού\seip_assignment_1_2026>curl http://localhost:8080/secure-config
{"status":"Authorized","injected_secret_suffix":"*****furt"}
```

Browser Engine



AI Reflection & Future Outlook

AI Integration:

For the purposes of this assignment, Claude by Anthropic was used to assist in completing the assignment tasks as a means of conceptual understanding, debugging and documentation. Specifically for each one:

- **Conceptual Understanding:** Before consulting the documentation required for implementing the relevant code (e.g. [Minikube Documentation](#)), AI was used to obtain high-level conceptual explanations, enabling a clearer understanding of the documentations. Additionally, AI explanations during implementation were often sufficient to avoid repeatedly consulting the documentation (e.g. explanation of the use of `livenessProbe` and `readinessProbe` in Task 3.3).
- **Debugging:** AI proved particularly useful in identifying small but important errors that could easily be overlooked. For example, it helped identify YAML structure issues such as steps not being correctly indented under the job section in `ci-cd.yaml` and that the `.github` directory missed the period at the beginning which prevented GitHub Actions from functioning.

- Documentation: AI was also utilized to assist in formatting the README.md file faster and to give suggestions on how it can be improved for the reader, such as highlighting certain parts (e.g. in Step 4, flagging the need for a second terminal, which, while initially appearing self-evident, could otherwise cause confusion for other users).

Utility Analysis and Friction Points:

For this assignment, Claude was more than capable of providing assistance regarding the aforementioned areas where it was used. It was particularly useful in explaining how Kubernetes functionalities are interconnected and their complementary roles. It was also very useful in interpreting terminal messages.

No significant hallucinations or incorrect configurations were encountered during this assignment, other than Claude incorrectly stating on multiple occasions when checking issues regarding the ci-cd.yaml file, that [actions/checkout@v6](#) did not exist and that the latest available versions only went up to v4, which demonstrates the model's limited real-time awareness. However, it should be mentioned that Claude would sometimes answer more than asked, or add unrequested content to the documentation, and as a result required close monitoring. This however, also proved helpful at times. For instance, while reviewing the correctness of secret.yaml's structure, Claude, without being prompted to, pointed out that base64 encoding is not real encryption and that the value could be decoded instantly with a single terminal command. This security insight was later used in the Future Architectural Outlook section of the report.

Future Architectural Outlook:

My suggestions regarding scaling this pipeline for a real-world enterprise system have to do with its current nature as a local pipeline, given the use of Minikube, which would need to be replaced by a production-grade cluster such as EKS (Elastic Kubernetes Service) provided by AWS.

Assuming that the real-world app is exposed on the public internet and reachable by all, an Ingress Controller could be used to properly route external traffic and replace the current port-forwarding workaround. Additionally, deployment management would be difficult to maintain manually. For this reason, integrating Argo CD and establishing a true GitOps workflow with it would significantly improve scalability. This would remove the need for the repetitive manual kubectl apply commands and automatically sync repository changes to the cluster.

Furthermore, the current base64 encoding approach for the Secrets manifest is not real encryption and can be easily decoded by anyone with access to the repository.

As such, in order to improve security a proper secrets management tool should be used like HashiCorp Vault.

Moving to monitoring, as highlighted in the DevOps lab slides, once a Kubernetes cluster scales to hundreds of containers, manually checking logs to detect failures, security breaches, or performance issues becomes completely infeasible. A natural next step would be to integrate Prometheus, expose the /metrics endpoint in the app, record real-time metrics and get notified when an event like a failure occurs. Optionally, Grafana can be integrated for visualization purposes. Moreover, the real-time metrics provided by Prometheus can also be used to automatically scale the number of running replicas based on real-time traffic, replacing the current static 3 replicas defined in the Deployment manifest. For this purpose, a HorizontalPodAutoscaler should be used for the scaling according to demand.

Finally, image scanning should be integrated into GitHub Actions workflow so vulnerabilities can automatically be found before they reach the cluster. Docker suggests using [tools that support VEX](#) (Vulnerability Exploitability eXchange) attestations (e.g. Docker Scout and Trivy).